UNIVERSITY OF St.Thomas

SEIS 606: Vibe Coding

Fall 2026

Paul Kaefer, Adjunct Professor

Tonight's Agenda

1. Framing & cautionary tales
2. "The three pillars" for deployment
3. Pair activity: break someone's app
4. Special presentation on Food4Thought
5. Discussion on humans in the loop for deployment
6. Pre-flight checklist
7. Logistics & expectations for project 1

Where are we at?

We've been building apps.

Goal: "shipping" and presenting.

Question of the day: can you vibe code something you'd trust with real users' data?

Cautionary tale #1

In 2025, Jason Lemkin who founded SaaStr used Replit. He instructed the agent to not make any further changes without his permission.

The agent deleted the entire database.

While the data wasn't lost entirely, this demonstrates the need to separate development & production. As well as the need for backups!

Source: "The real agent risk that Replit's database deletion revealed." October 9, 2026.

Cautionary tale #2

A vulnerability in Loveable's row-level security (RLS) policies allowed attackers to query exposed databases and read/write sensitive data from any table.

This demonstrates the need for secure database policies & authentication requirements.

Source: "CVE-2025-48757: Lovable Information Disclosure Vulnerability." January 22, 2026.

Cautionary tale #3

Research has shown that one in five organizations build on vibe coding platforms prone to several potential issues:

1. Authentication logic in browser (passwords visible in JavaScript code!)
2. API keys/secrets visible client-side
3. Database tables visible to anyone (see previous slide!)
4. Internal applications that are public-facing without authentication
   - Search for "lovable.app" found numerous cases of proprietary/sensitive info being leaked

Source: "Wiz Research Discovers One in Five Organizations Exposed to Systemic Risks in Vibe-Coded Applications - Here's How to Secure Them." September 18, 2025.

Discussion

Have you read about other cases?

- Chris said China issued digital citizenships to AI personalities.

What else can we learn (or have we learned) from these?

- Tools like Vercel, Claude are now “smart” enough to recognize keys in chat & will put them in the right place. And/or tell you to regenerate the key!

The "three pillars" for deployment

Databases: *“Who can read and write each table, and how do I know?”*

- Row-level security
- Backups
- Development vs. production
- What do we do when we've exhausted the "free tier"?

Security/auth: *“What happens if someone skips my UI and hits the API directly?”*

- Secrets in the repository or visible in the frontend
- client-side-only checks/rolling your own auth.

Uptime/downtime: *“How will I know it’s down, and what do I do then?”*

- Hosting limits
- Monitoring & "who gets paged?"

Pair activity: break someone's app

Similar to week 4, pair off and try to find vulnerabilities

- Exposed keys or passwords in code?
- Unauthenticated URLs?
- Open database rules?

Don't have to be an expert on security!

- Can ask AI to review for security & (then you) analyze the results.

Deployment: human-in-the-loop

- **Unknown unknowns:** You can only prompt for risks you know exist. The AI answers the question you asked, not the one you should have asked.
- **Works vs. safe:** Generated code is rewarded for working. An app can pass every demo and be wide open.
- **Defender's asymmetry:** An attacker needs one gap, while you need to close all of them.
- **Accountability:** When the breach happens, someone must answer for it, and it won't be the model.
- **A fair counterpoint to raise yourself:** Humans write insecure code all the time. The real contrast is between *reviewed, understood* code and *unreviewed* code. AI can be a useful reviewer, and the human role is judgment and ownership.

For discussion…

What's something you built (in your app) so far that you couldn't fully explain?

- Graphics on site
- Website design
- Sample data/statistics → used for testing/examples
  - Further, finding patterns/combinations we wouldn’t have thought of
- Vulnerabilities
- Specific details about modern infrastructure (PDF uploads, but also hosted PDF files → like Google Drive/any website)

For discussion...

At what point does an app need a human expert, and what's the signal?

- Whenever you run into a wall of knowledge or expertise
- Any time you lose ~~a client/database~~ something
- You understand your userbase better than your agent(s) do!
  - Designing something to cater to your users: your input is needed
- User feedback/maintenance

For discussion…

Would you put your own data in an app built the way yours was?

- Francis answered “no!” right away
- John mentioned dogfooding

Scott's demo

- Food4Thought, working!?

- Some questions:
  - Steps to deploy
  - Costs to deploy

Logistics for next week

- Working app
  - Prepare a backup plan (recording/screenshots)

- Presentation in about 5-7 minutes + discussion/letting the crowd explore?
  - 1 minute overview
  - 2-3 minute demo
  - 2 minutes on testing findings
  - 1 minute on gap analysis/future work

- Willing to take volunteers for order, but otherwise will randomize…

Pre-flight checklist

- ☐ Make sure no passwords/API keys or other secrets are visible when presenting!
- ☐ Think about what you would fix before "real users" engage with the app.

- ☐ See two versions of a checklist (generated using Claude) on Canvas
  - Not *requiring* you to turn it in, but go through it & see how your app fares.

UNIVERSITY OF St.Thomas

I'm looking forward to project presentations next week!